A SYNOPSIS ON

Campus Route & Shortest-Path Navigator

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)
(BSC IT)

Submitted by:

SUMEET YADAV **Student ID: 2510090039**

Under the Guidance of

Mr. Mohd Shuaib



Department of Computer Applications
Graphic Era (Deemed to be University)
Dehradun, Uttarakhand
September-2026

CANDIDATE'S DECLARATION

I/we hereby certify that the work which is being presented in the Synopsis entitled "Campus Route & Shortest-Path Navigator" in partial fulfillment of the requirements for the award of the Degree of Bachelor of Science (Information Technology) in the Department of Computer Applications of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision of [GUIDE NAME], [Designation], Department of Computer Applications, Graphic Era (Deemed to be University), Dehradun.

Name: Sumeet Yadav Student ID: 2510090039

signature

The above mentioned student shall be working under the supervision of the undersigned on the "Campus Route & Shortest-Path Navigator"

Signature	Signature
Supervisor	Head of the Department

Internal Evaluation (By Project Committee)

Status of the Synopsis: Accepted / Rejected

Any Comments:

Name of the Committee Members: Signature with Date

1.

2.

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	**Introduction and Problem Statement** Introduces the need for an interactive shortest-path visualization tool built on a real campus/road network. Explains how the system models locations as nodes and paths as weighted edges for route computation. States the problem of visualizing and comparing pathfinding algorithms on real-world map data.	4-6
Chapter 2	**Background/ Literature Survey** Reviews Dijkstra's Algorithm and the A* Algorithm as foundational shortest-path search techniques. Surveys OpenStreetMap-based tools such as OSMnx, Leaflet.js, and existing routing libraries. Identifies the gap in tools that visually animate and compare algorithm behaviour on real data.	6-7
Chapter 3	**Objectives** Lists the goal of modelling a real campus/city area as a graph using actual OpenStreetMap data. States the aim of integrating existing pathfinding libraries to compute Dijkstra and A* routes. Defines the objective of animating node exploration and comparing algorithm efficiency visually.	7
Chapter 4	**Hardware and Software Requirements** Specifies the minimum hardware needed to run and develop the web-based navigator application. Lists the software stack including mapping libraries, graph libraries, and the development tools. Covers the runtime environment required for fetching and rendering real-world map data.	8
Chapter 5	**Possible Approach/ Algorithms** Describes the stage-wise approach from data acquisition to graph construction and rendering. Explains how Dijkstra and A* are executed using existing libraries on the extracted road network. Details the wavefront animation and the node-visit comparison between the two algorithms.	9-10
	References	11

Chapter 1

Introduction and Problem Statement

In the following sections, a brief introduction and the problem statement for the work has been included.

1.1 Introduction

Navigating a large college campus or a locality is a routine but often inefficient task for students, staff, and visitors, who typically rely on static maps or word-of-mouth directions that do not account for the shortest or most efficient path between two points. The problem of finding the shortest path between two locations is a classical problem in computer science and can be solved effectively using graph-based shortest-path algorithms.

The proposed Campus Route & Shortest-Path Navigator is a web-based application that models a chosen real-world area — such as a university campus or a section of a city — as a weighted graph, where road/path intersections and landmarks are represented as nodes and the roads connecting them are represented as weighted edges, with the weight corresponding to real physical distance. The graph is not invented or simplified by hand; instead, it is built directly from real-world OpenStreetMap (OSM) road-network data, using the OSMnx Python library, which downloads the actual street/path layout of a specified place and converts it into a ready-to-use graph structure.

The application allows a user to select a source and a destination on an interactive map and computes the shortest path between the two points using well-established pathfinding algorithms, namely Dijkstra's Algorithm and the A* (A-Star) Algorithm. Rather than implementing these algorithms from scratch, the project focuses on integrating existing, well-tested libraries for graph representation and pathfinding (such as NetworkX/OSMnx on the data-preparation side, and ngraph.path or graphology-shortest-path on the web visualization side), and on building an interactive visualization layer on top of them.

A key feature of the application is the visual “algorithm race”: as the shortest path is being computed, the nodes visited by the algorithm are animated on the map in real time, giving the appearance of a wavefront expanding outward from the source until it reaches the destination. The application also allows the user to toggle between Dijkstra and A* and compare the number of nodes each algorithm visits before finding the optimal path on the

same real road network, offering an intuitive, visual understanding of how informed (heuristic-based) search differs from uninformed search.

Figure 1.1 Overview of the AI-assisted route computation pipeline

1.2 Problem Statement

Most people navigating an unfamiliar campus or locality have no easy way to visualize how a shortest-path algorithm actually works on a real road network, nor an interactive tool to compare the efficiency of different pathfinding strategies. Standard navigation applications show only the final route, without offering any insight into the underlying search process, the number of nodes explored, or the difference in performance between algorithms.

There is a need for an educational and interactive tool that can take a real-world road network (rather than a hand-made or invented one), allow a user to pick any two points on it, and visually demonstrate how different shortest-path algorithms explore the graph and arrive at a solution, while also presenting a quantitative comparison, such as node-visit counts, between the algorithms used.

The proposed Campus Route & Shortest-Path Navigator aims to solve this problem by combining a real OpenStreetMap-based road network with existing graph and pathfinding libraries to compute and animate the shortest path, making the process of pathfinding both visible and comparable across algorithms on genuine geographic data.

Chapter 2

Background/ Literature Survey

Shortest-path algorithms are among the most extensively studied problems in graph theory and computer science, with applications ranging from road navigation systems and network routing to game AI and logistics planning. A survey of the relevant literature and existing tools is presented below.

2.1 Shortest-Path Algorithms

Dijkstra's Algorithm, proposed by Edsger W. Dijkstra in 1959, is a classical greedy algorithm used to find the shortest path from a single source node to all other nodes in a weighted graph with non-negative edge weights. It works by repeatedly selecting the unvisited node with the smallest known distance from the source and relaxing the distances

of its neighbouring nodes, continuing until the destination is reached or all reachable nodes are processed.

The A* (A-Star) Algorithm, introduced by Hart, Nilsson, and Raphael in 1968, builds upon Dijkstra's approach by introducing a heuristic function that estimates the distance from a given node to the destination, allowing the algorithm to prioritize exploring nodes that are more likely to lead towards the goal. This makes A* generally more efficient than Dijkstra in practice, as it typically visits fewer nodes to reach the same destination, particularly when a good heuristic (such as straight-line/Euclidean or Haversine distance) is available, as is the case with geographic coordinates on a real map.

2.2 Real-World Map Data and Tools

OpenStreetMap (OSM) is a free, editable, crowd-sourced map of the world that provides detailed, real road-network data for virtually any location. Tools such as OSMnx (built on top of NetworkX and GeoPandas) allow a real drivable, walkable, or bikeable street network to be downloaded for any named place, address, or bounding box, and automatically converted into a graph object with accurately weighted edges (based on real segment lengths), removing the need to manually construct or invent a dataset.

On the web front end, Leaflet.js is a widely used, lightweight JavaScript library for rendering interactive maps using OSM tiles directly in the browser. Libraries such as ngraph.graph and ngraph.path, or graphology together with graphology-shortest-path, provide ready-made, well-tested implementations of graph data structures and pathfinding algorithms (including Dijkstra and A*) that can be integrated directly into a JavaScript/web application.

2.3 Existing Systems and Research Gap

Several existing navigation platforms, such as Google Maps and OpenRouteService, already make use of shortest-path algorithms internally on real-world data, but they generally do not expose the internal working of the algorithm to the end user. There is comparatively little tooling aimed specifically at visualizing and comparing the step-by-step behaviour of these algorithms on a real, extracted road network in an educational and interactive manner.

Based on this survey of existing literature and tools, the proposed project focuses on combining a real map interface (Leaflet.js with OSM tiles) with a real-world road-network dataset (extracted using OSMnx from OpenStreetMap) and existing JavaScript pathfinding

libraries, to build an interactive tool that visually animates and compares the behaviour of Dijkstra's Algorithm and the A* Algorithm on genuine geographic data.

Chapter 3

Objectives

The objectives of the proposed work are as follows:

- To extract a real-world road network for a chosen campus/locality from OpenStreetMap using OSMnx and convert it into a weighted graph of nodes (intersections/landmarks) and edges (road segments).
- To render this real road network on an interactive map using Leaflet.js and OSM tiles, so users can visually see actual roads and landmarks rather than an abstract or invented layout.
- To integrate existing JavaScript graph/pathfinding libraries to compute the shortest path between a user-selected source and destination using Dijkstra's Algorithm and the A* Algorithm on the real graph.
- To visually animate the node-expansion process of each algorithm as a live "wavefront" on the map, so the user can see how each algorithm searches the real network before settling on the shortest path.
- To provide a toggle/comparison feature that displays the number of nodes visited by Dijkstra versus A* for the same source-destination pair on the real dataset, enabling a direct visual and quantitative comparison of the two algorithms.

Chapter 4

Hardware and Software Requirements

4.1 Hardware Requirements

Sl. No	Name of the Hardware	Specification
1	Processor	Core i3/i5 or equivalent
2	RAM	8 GB – 12 GB
3	Hard disk/SSD	5–10 GB
4	Internet connection	Required for downloading OSM map data and tiles

4.2 Software Requirements

Sl. No	Name of the Software	Specification
1	Operating system	Windows/Linux
2	Programming languages	JavaScript, HTML, CSS, Python (data preparation)
3	Map & visualization	Leaflet.js, OpenStreetMap tiles
4	Real-world data extraction	OSMnx, NetworkX (Python)
5	Graph & pathfinding library (web)	ngraph.graph / ngraph.path (or graphology + graphology-shortest-path)
6	Package manager	npm (Node.js), pip (Python)
7	Front-end tooling	Vite / plain HTML-CSS-JS bundle
8	Dataset	Real-world road network extracted from OpenStreetMap (GraphML/GeoJSON)
9	Development environment	VS Code

Chapter 5

Possible Approach/ Algorithms

The proposed Campus Route & Shortest-Path Navigator will be built primarily using HTML, CSS, and JavaScript for the interactive front end, along with existing, well-tested libraries for mapping and pathfinding, rather than implementing the underlying graph algorithms from scratch. The road network used for path computation will be a real-world dataset extracted directly from OpenStreetMap using the OSMnx Python library, ensuring that the graph reflects an actual campus/locality rather than an invented layout.

The whole process can be split into the following stages:

5.1 Real-World Data Extraction

A specific place (a university campus area or a chosen locality) is selected, and its road network is downloaded using OSMnx, which queries OpenStreetMap's data and returns a graph of real intersections (nodes, with latitude/longitude) and real road segments (edges, weighted by their true length in metres). This graph is exported (e.g., as GraphML or GeoJSON) for use in the web application.

5.2 Map Rendering

Leaflet.js is used along with OpenStreetMap tiles to render an interactive map in the browser. The real nodes and edges extracted in the previous stage are plotted on this map, so the visualization directly overlays the actual road layout of the chosen area.

5.3 Graph Construction (Client Side)

The exported real-world graph data is loaded into the browser and used to build a graph object with an existing JavaScript graph library (such as ngraph.graph or graphology), with each real node and weighted edge added to the graph object provided by the library.

5.4 Source and Destination Selection

The user clicks on two points on the map — one as the source and one as the destination — which are snapped to the nearest real graph nodes and passed to the pathfinding module.

5.5 Shortest-Path Computation

An existing pathfinding library (such as ngraph.path, which provides both Dijkstra's Algorithm and A*) is used to compute the shortest path between the source and destination nodes on the real graph. The library also exposes the order in which nodes are visited/expanded during the search, which is used for the animation step. The cosine/heuristic distance used by A* is computed from real latitude/longitude coordinates.

Figure 5.1 Wavefront expansion during shortest-path computation on the real road network

5.6 Wavefront Animation

Using the sequence of visited nodes returned by the pathfinding library, the application animates each visited node on the map in the order they were explored, creating a live "wavefront" effect that visually expands outward from the source before settling on the final shortest path, which is then highlighted distinctly on the real map.

5.7 Algorithm Comparison (Dijkstra vs A*)

A toggle control allows the user to switch between Dijkstra's Algorithm and A* for the same source-destination pair on the real road network. The application records and displays the number of nodes visited by each algorithm, along with the resulting path length/distance, allowing a direct visual and quantitative "algorithm race" comparison between the two approaches on genuine geographic data.

5.8 Result Display

The final output presents the computed shortest path drawn on the real map, the total distance of the path in metres, the sequence of streets/landmarks along the route, and a side-by-side comparison of node-visit counts between Dijkstra and A* for that particular query.

High-Level Steps

- Select a target area (campus/locality) and download its real road network using OSMnx from OpenStreetMap;
- Export the extracted graph (nodes with coordinates, edges with real distances) for use in the browser;
- Render the OpenStreetMap-based map using Leaflet.js and plot the real nodes and edges on it;
- Build a graph object in the browser from the extracted data using an existing graph library (e.g., ngraph.graph);
- Let the user click a source node and a destination node on the map;
- Run Dijkstra's Algorithm using the chosen pathfinding library and record the visited-node sequence and final path;
- Run the A* Algorithm using the same library and record its visited-node sequence and final path;
- Animate the visited-node sequence for the selected algorithm as an expanding wavefront on the map;
- Highlight the final shortest path once the destination node is reached;
- Display the total path distance and the sequence of streets/landmarks along the route;
- On toggling the algorithm, re-run the computation and update the animation, path, and node-visit count accordingly;
- Display a side-by-side comparison of node-visit counts and path distance for Dijkstra vs A*;

References

[1] E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[2] P. E. Hart, N. J. Nilsson and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[3] G. Boeing, “OSMnx: New Methods for Acquiring, Constructing, Analyzing, and Visualizing Complex Street Networks,” Computers, Environment and Urban Systems, vol. 65, pp. 126-139, 2017.

[4] OpenStreetMap Foundation, “OpenStreetMap,” [Online]. Accessed on 24th September 2026: https://www.openstreetmap.org.

[5] Leaflet, “Leaflet — an open-source JavaScript library for interactive maps,” [Online]. Accessed on 24th September 2026: https://leafletjs.com.

[6] A. Rembish, “ngraph.path — Path finding in a graph,” [Online]. Accessed on 24th September 2026: https://github.com/anvaka/ngraph.path.

[7] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, “Introduction to Algorithms,” 3rd ed., MIT Press, 2009.